

High Level Design

Members:

- Benjamin Strizakov
- Yara Zeineh
- Carol Elasmarr
- Omer Samara
- Bayan Kheir
- Kareem Araide

Introduction:

GhostHouses is a system for logging and tracking abandoned buildings in the city of Haifa. The project's goal is to replace the current manual process of managing information about abandoned buildings by Haifa municipality. The system will be integrated with existing IT services for user authentication and connecting with a database.

1. User Stories

1.1 User Story List

Authentication & Roles

US-1 – As an admin, I want a clear role model (Viewer, User–Editor, Admin) so that access and responsibilities are unambiguous.

US-2 – As an admin, I want users to log in with 2FA so that only authorized people can use the system.

US-3 – As an admin, I want to set municipality user permissions to view/edit/can't access so that system access is controlled.

Buildings (Read/Search)

US-4 – As a viewer, I want to view building details so that I can review its current status.

US-5 – As a viewer, I want to search and filter buildings (street, house no., nickname, status/category, area) so that I can quickly find relevant records.

US-6 – As a viewer, I want to open a building page/card with photos so that I can understand the property at a glance.

US-7 – As a viewer, I want to produce a building card image to include in presentations.

Buildings (Edit)

US-8 – As an editor, I want to add new buildings so that the database stays current.

US-9 – As an editor, I want to update existing buildings so that errors and changes are reflected promptly.

US-10 – As an editor, I want to remove buildings (with safeguards) so that obsolete or duplicate entries are cleaned up.

Logs & Audit

US-11 – As a viewer, I want to view a building's change log so that I can understand its history.

US-12 – As a viewer, I want to search and filter logs so that I can locate specific events quickly.

US-13 – As an editor, I want to add or edit log entries for a building so that important actions are recorded.

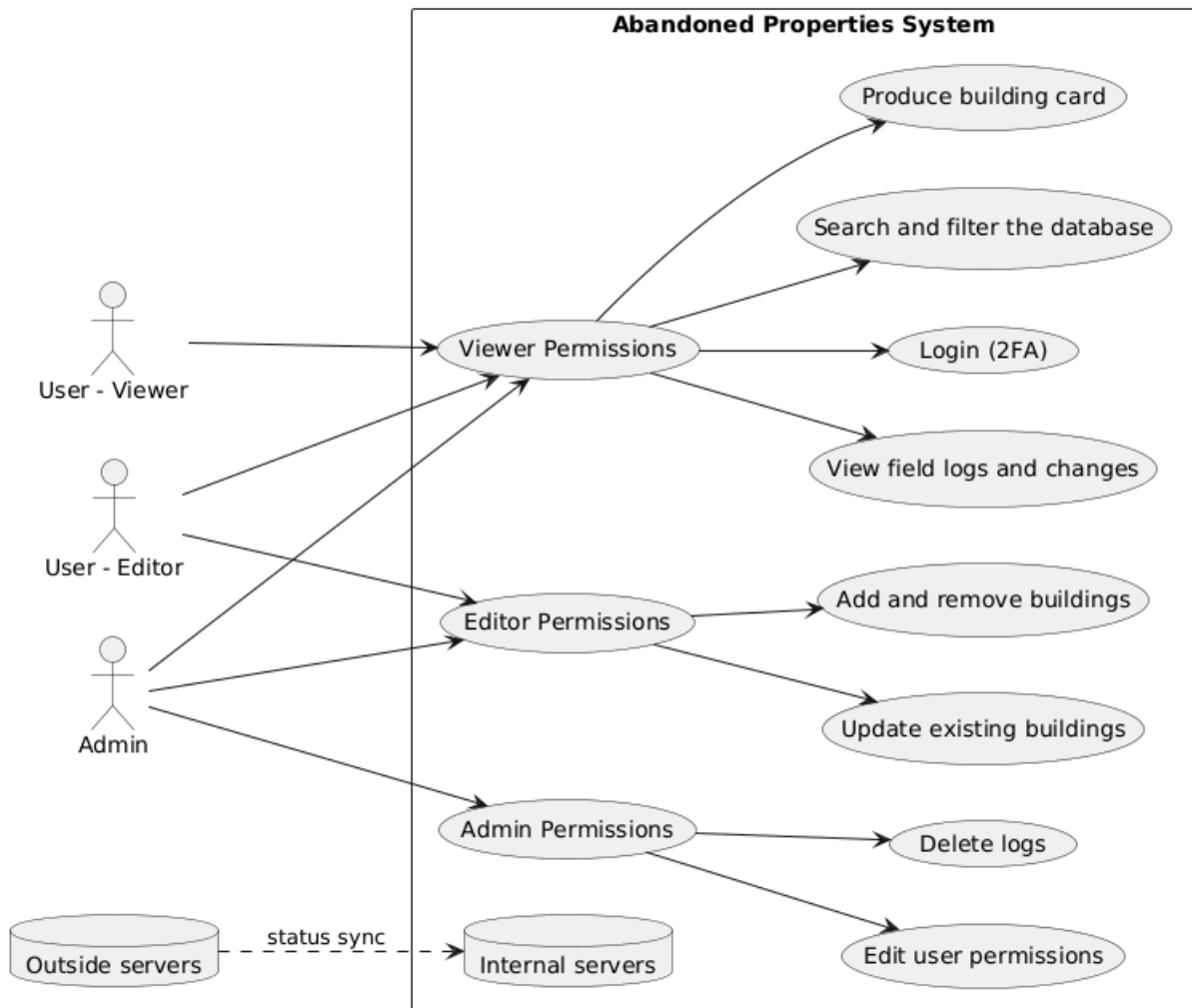
US-14 – As an admin, I want to delete log entries so that sensitive or erroneous logs can be corrected while preserving accountability.

US-15 – As a viewer, I want a full audit trail of who changed what and when so that compliance requirements are met.

Data Synchronization

US-16 – As an editor, I want status/data to sync with external municipality systems so that data remains consistent across systems.

1.2 Use Case Diagram

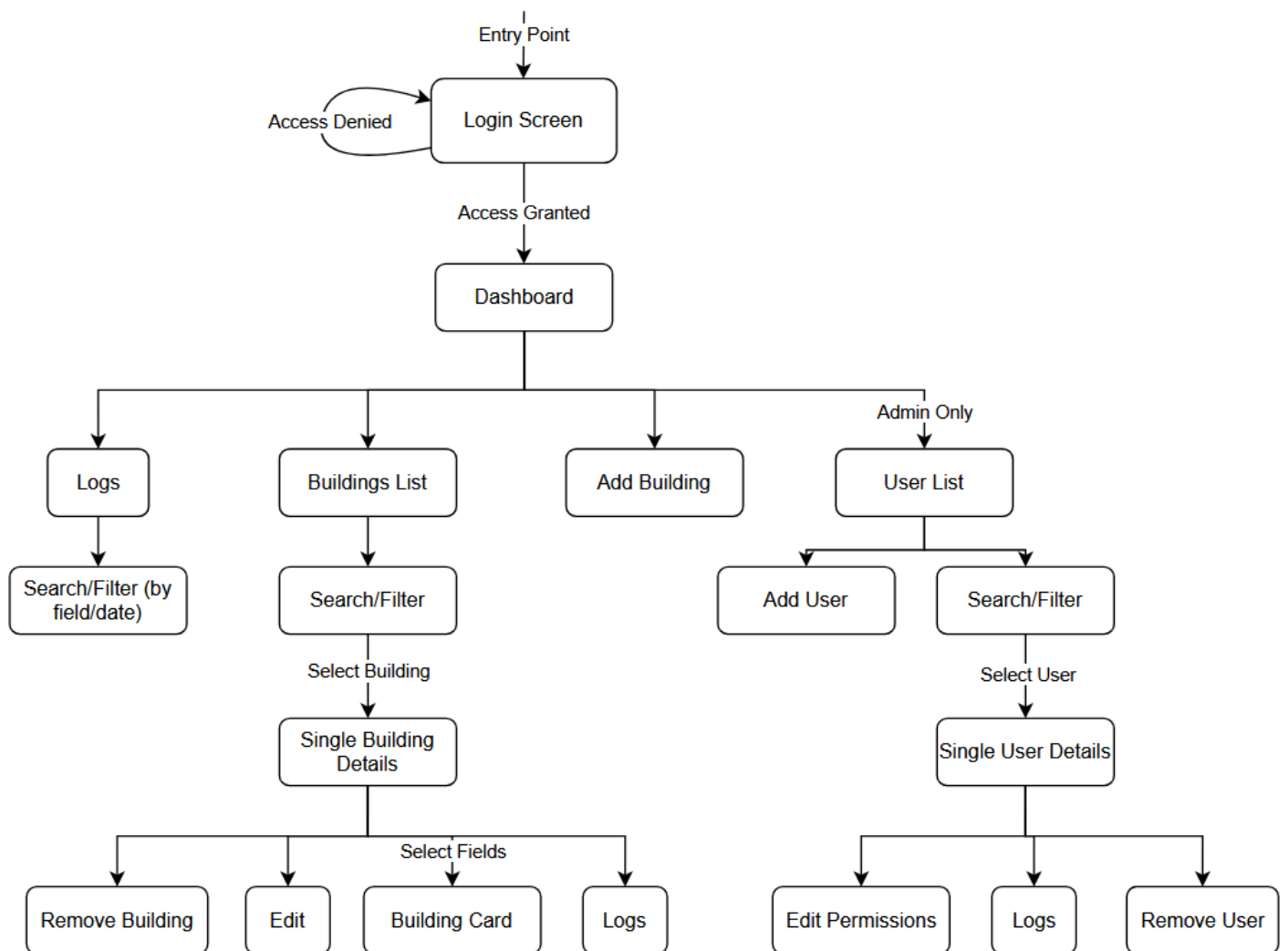


There will be three roles:

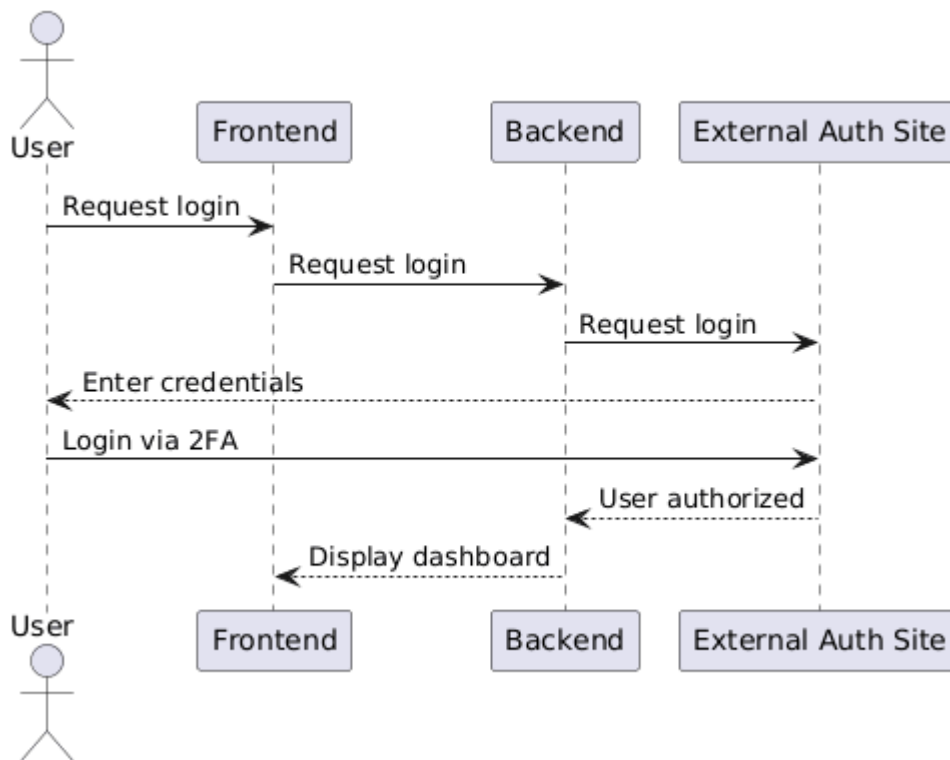
1. Viewer - a user who only has access to view logs and building details, and produce building cards.
2. Editor - a user who has all the Viewer permission, and in addition will have access to edit building details, and add/remove them.
3. Admin - a user who has all the Editor permission, and in addition will be able to edit user permission, and delete logs.

1.3 Sequence Diagram

Website structure

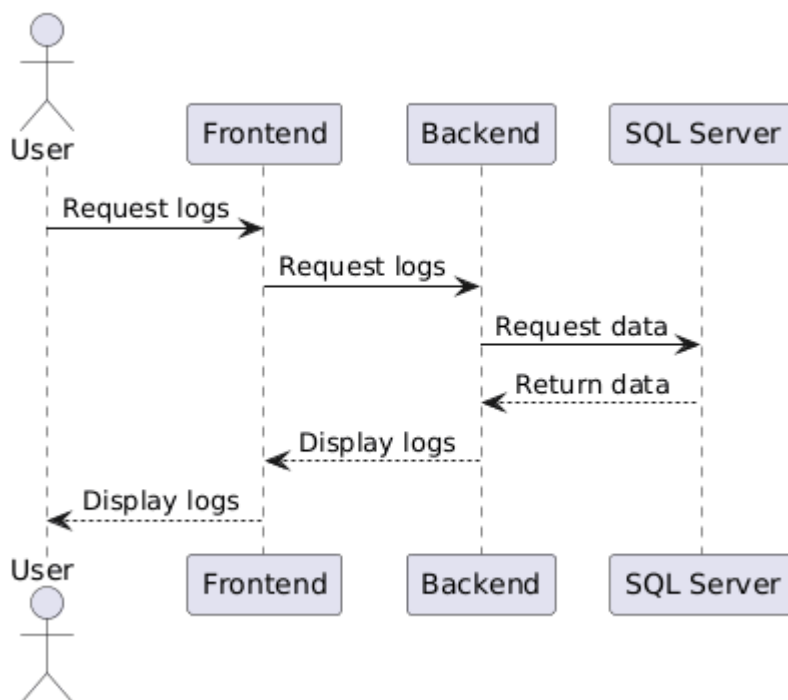


Login (2FA)



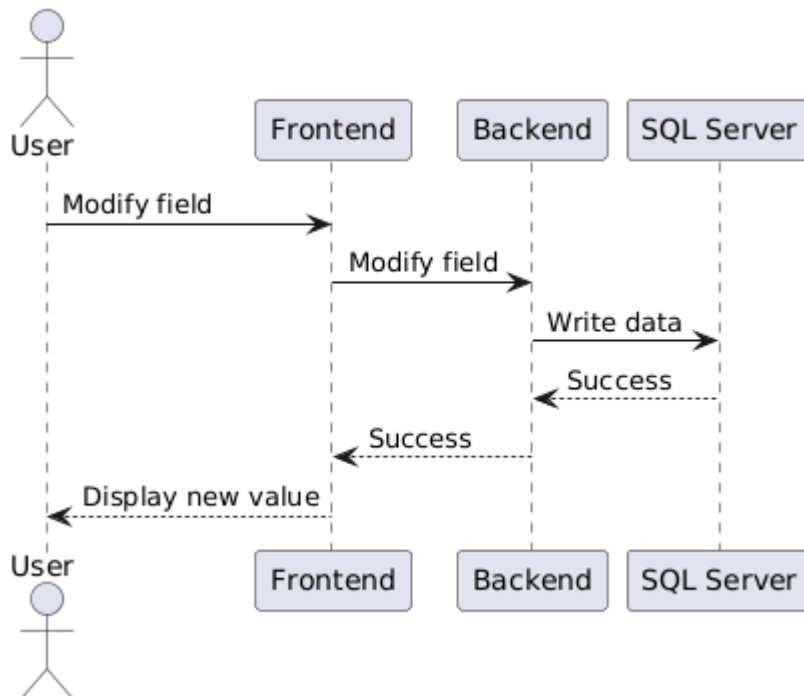
When the user enters their credentials, a login request will be propagated from the frontend to the authentication server. The authentication server will authenticate the user, and the GUI will display either “access denied” or the system dashboard on a successful authentication.

View Field Logs and Changes



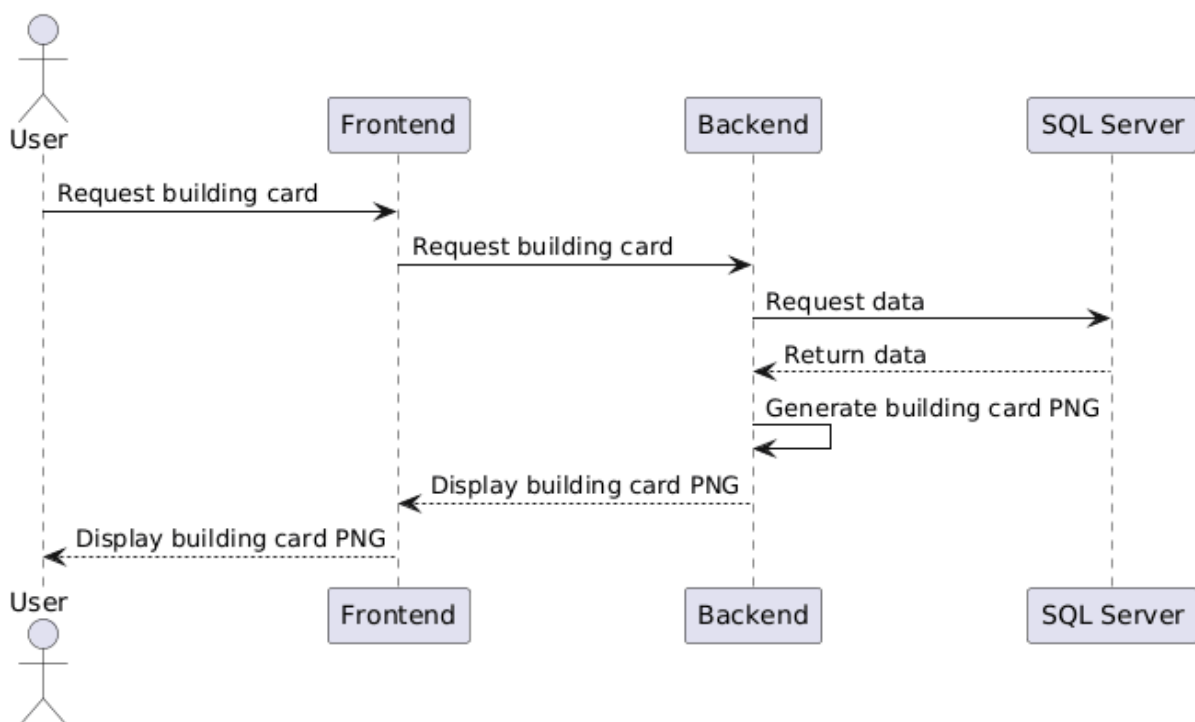
When the user requests logs of a certain building/field, the request will be propagated to the SQL server, and the relevant details will be displayed in the GUI.

Update Entries



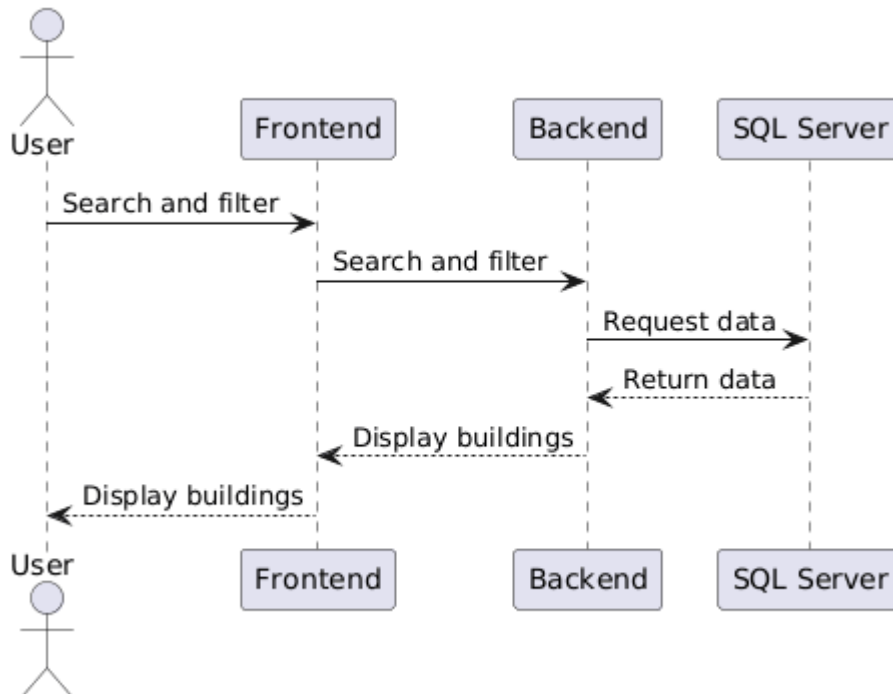
When the user modifies a building field,, the change will be propagated to the SQL server, and the GUI will update accordingly.

Produce Building Card



When the user requests a building card, the request will be propagated to the backend, which will get the relevant data from the SQL server. The backend will then compile the data into a PNG, which will be displayed to the user.

Search and Filter



When the user enters a search query, the request will be propagated to the backend, which will gather the data from the SQL server and display it to the user via the frontend.

1.4 Break down to tasks

Authentication & Roles

US-1: As an admin, I want a clear role model (Viewer, Editor, Admin) so that access and responsibilities are unambiguous.

US-2: As an admin, I want users to log in with 2FA so that only authorized people can use the system.

US-3: As an admin, I want to set municipality user permissions to view/edit/no access.

- **Task A1:** Define and document role hierarchy (Viewer, Editor, Admin) - maps to US-1.
- **Task A2:** Develop login interface and integrate 2FA logic - maps to US-2.
- **Task A3:** Implement user permission management interface (assign roles) - maps to US-3.
- **Task A4:** Backend authorization middleware enforcing roles and permissions - maps to US-1, US-3.

Buildings (Read/Search)

US-4: As a viewer, I want to view building details so that I can review its current status.

US-5: As a viewer, I want to search and filter buildings (street, house no., nickname, status/category, area) so that I can quickly find relevant records.

US-6: As a viewer, I want to open a building page/card with photos so that I can understand the property at a glance.

US-7: As a viewer, I want building cards to include data fetched from external municipality systems so that the information is comprehensive.

- **Task B1:** Design building list UI with search/filter controls - maps to US-5.
- **Task B2:** Build building detail/card UI showing status, photos - maps to US-4, US-6.
- **Task B3:** Backend endpoint for building list and filters - maps to US-5.
- **Task B4:** Backend endpoint for building card including external data integration - maps to US-7.
- **Task B5:** Integrate external data sources (GIS, water, tax) for building card - maps to US-7. (optional)

Buildings (Edit)

US-8: As a user–editor, I want to add new buildings so that the database stays current.

US-9: As a user–editor, I want to update existing buildings so that errors and changes are reflected promptly.

US-10: As a user–editor, I want to remove buildings (with safeguards) so that obsolete or duplicate entries are cleaned up.

- **Task C1:** Design UI for add/update/remove building operations - maps to US-8, US-9, US-10.
- **Task C2:** Backend API for create/update/delete building records - maps to US-8, US-9, US-10.
- **Task C3:** Audit service capturing all building edits/deletions - maps to US-9, US-10.
- **Task C4:** Safeguards and validation logic for removing buildings (e.g., open inspections) - maps to US-10.

Logs & Audit

US-11: As a viewer, I want to view a building's change log so that I can understand its history.

US-12: As a viewer, I want to search and filter logs so that I can locate specific events quickly.

US-13: As a user–editor, I want to add or edit log entries for a building so that

important actions are recorded.

US-14: As an admin, I want to delete log entries so that sensitive or erroneous logs can be corrected while preserving accountability.

US-15: As a viewer, I want a full audit trail of who changed what and when so that compliance requirements are met.

- **Task D1:** Design log view UI (list, filters, details) - maps to US-11, US-12.
- **Task D2:** Backend endpoint to fetch logs by building, user, date - maps to US-11, US-12.
- **Task D3:** UI and API for adding/editing logs - maps to US-13.
- **Task D4:** UI and API for deleting logs (admin only) + audit recording - maps to US-14.
- **Task D5:** Design and implement audit trail storage for all entity changes (who, what, when) - maps to US-15.

Data Synchronization

US-16: As a user–editor, I want status/data to sync with external municipality systems so that data remains consistent across systems.

- **Task E1:** Identify and document external data sources and schema (GIS, CRM-106, Water, Tax) - maps to US-16.
- **Task E2:** Develop ingestion services to import/update data from external systems - maps to US-16.
- **Task E3:** Schedule and automate recurring sync jobs + error handling/notifications - maps to US-16.
- **Task E4:** Map external fields to internal schema, update data models as needed - maps to US-16.

1.5 Task acceptance criteria

Authentication & Roles

Task A1 – Define and document role hierarchy (Viewer, Editor, Admin) → US-1

- **Given** the system design phase is active
- **When** roles are defined and documented in specifications
- **Then** each role has a clear scope of permissions and responsibilities without overlap

Task A2 – Develop login interface and integrate 2FA → US-2

- **Given** a user is on the login screen
- **When** valid credentials are entered and submitted
- **Then** an OTP is sent and successful 2FA login redirects to the dashboard

- **Given** an invalid OTP is entered
- **Then** the system displays an error and does not grant access

Task A3 – Implement user permission management interface (assign roles) → US-3

- **Given** an admin is logged in
- **When** they open the User Management page
- **Then** they can assign, update, or remove roles and permissions successfully

Task A4 – Backend authorization middleware → US-1, US-3

- **Given** any user request to the API
- **When** a restricted action is attempted
- **Then** only authorized roles may execute it, and unauthorized users receive an error

Buildings (Read / Search)

Task B1 – Design building list UI with search / filter → US-5

- **Given** the user is on the building list page
- **When** they apply filters (e.g., street or area)
- **Then** the UI displays only matching records

Task B2 – Build building detail / card UI → US-4, US-6

- **Given** the user selects a building from the list
- **When** the building page loads
- **Then** status, photos, and metadata are displayed correctly

Task B3 – Backend endpoint for building list and filters → US-5

- **Given** the frontend requests the building list with parameters
- **When** the API query runs
- **Then** results are returned with filtering and pagination

Task B4 – Backend endpoint for building card (integration) → US-7

- **Given** the frontend requests a building card
- **When** the API fetches external municipality data
- **Then** the response combines internal and external information

Task B5 – Integrate external data sources (GIS, Water, Tax) (optional)→ US-7

- **Given** external systems are reachable
- **When** a sync or card fetch is triggered
- **Then** all external fields are retrieved and merged without conflicts

Buildings (Edit)

Task C1 – Design UI for add / update / remove buildings → US-8–US-10

- **Given** an editor is on the dashboard
- **When** they select “Add”, “Edit”, or “Remove”
- **Then** the UI opens the appropriate form and validates inputs

Task C2 – Backend API for create / update / delete records → US-8–US-10

- **Given** valid JSON requests are sent to the API
- **When** operations succeed
- **Then** the database reflects the changes and returns status codes

Task C3 – Audit service for building edits → US-9, US-10

- **Given** a record is modified or removed
- **When** the change occurs
- **Then** a log entry records the user, time, and action

Task C4 – Safeguards and validation logic for deletions → US-10

- **Given** a user attempts to remove a building
- **When** the deletion is accidental/unauthorized
- **Then** the system prevents deletion

Logs & Audit

Task D1 – Design log view UI (list / filters / details) → US-11, US-12

- **Given** a user opens the Logs page
- **When** filters are applied
- **Then** the list updates and shows the matching entries

Task D2 – Backend endpoint to fetch logs → US-11, US-12

- **Given** a query with filters (user, building, date...)
- **When** the request is processed
- **Then** the response returns only matching log records

Task D3 – UI and API for adding / editing logs → US-13

- **Given** a user opens a building’s log tab
- **When** they change a logged field
- **Then** the log is automatically stored and visible

Task D4 – Admin delete log capability → US-14

- **Given** an admin selects a log for deletion
- **When** the request is confirmed
- **Then** the log is removed and the action recorded in audit trail

Task D5 – Audit trail storage → US-15

- **Given** any entity update occurs
- **When** the system saves changes
- **Then** a complete record of who, what, and when is stored persistently

Data Synchronization

Task E1 – Identify and document external data sources → US-16

- **Given** integration requirements are gathered
- **When** data sources are analyzed
- **Then** each system's schema and fields are documented clearly

Task E2 – Develop ingestion services → US-16

- **Given** external systems provide data feeds
- **When** the ingestion runs
- **Then** records are pulled and stored without duplicates

Task E3 – Automate recurring sync jobs → US-16

- **Given** the scheduler is configured
- **When** the time based job scheduler executes
- **Then** new and updated records are synced automatically

Task E4 – Map external fields to internal schema → US-16

- **Given** a new data mapping is defined
- **When** data is transferred
- **Then** all fields are aligned and stored in the correct format

2. Architecture

2.1 Alternatives & Discussion

Backend Alternatives:

Java

- **Pros:**
 - Mature, widely adopted language with large community support.
 - Robust frameworks like Spring Boot.
 - Strong multithreading and performance capabilities.
 - Slightly more familiar to the team.
- **Cons:**
 - Longer development cycles compared to modern alternatives.
 - A heavier runtime environment can complicate deployment.
 - Integration with Windows-based municipal IT systems may be less seamless.

C# (.NET)

- **Pros:**
 - Fast development with tools like Visual Studio.
 - Excellent integration with Windows-based municipal IT infrastructure.
 - Strong performance and minimal runtime overhead.
 - Rich libraries for data management, security, and API development.
 - Modern, maintainable language suitable for enterprise applications.
- **Cons:**
 - Less portable to non-Windows environments (minor concern if infrastructure is Windows-based).

Decision: C# is preferred for its productivity, maintainability, and compatibility with municipal IT systems.

Frontend Alternatives:

Option A: React

Pros:

- Large and mature ecosystem with long-term support.
- Fast development due to many ready-made components.
- Excellent documentation and community support.
- Fully supports RTL and Hebrew localization.
- Highly modular and easy to maintain.
- Strong testing ecosystem compatible with TDD workflows.

Cons:

- Requires JavaScript/TypeScript proficiency.
- Build tooling (Webpack/Vite/etc.) adds some complexity.

Option B: Python (Flask/Django templating or Python-based UI generators)**Pros:**

- Simple syntax; good for quick prototypes.
- Easy backend integration if the backend were Python.
- The team is familiar with Python.

Cons:

- Not suitable for modern, interactive web GUIs.
- Very limited component ecosystem compared to React.
- Poor support for rich dashboards, maps, and dynamic UI.
- Weaker RTL/Hebrew support in templating engines.
- Slower development for complex municipal frontends.

Conclusion (Frontend):

React provides far faster development, modern UI capabilities, and excellent Hebrew/RTL support. Python-based frontends are not appropriate for a production municipal system. Therefore, React is selected as the frontend technology.

Conclusion

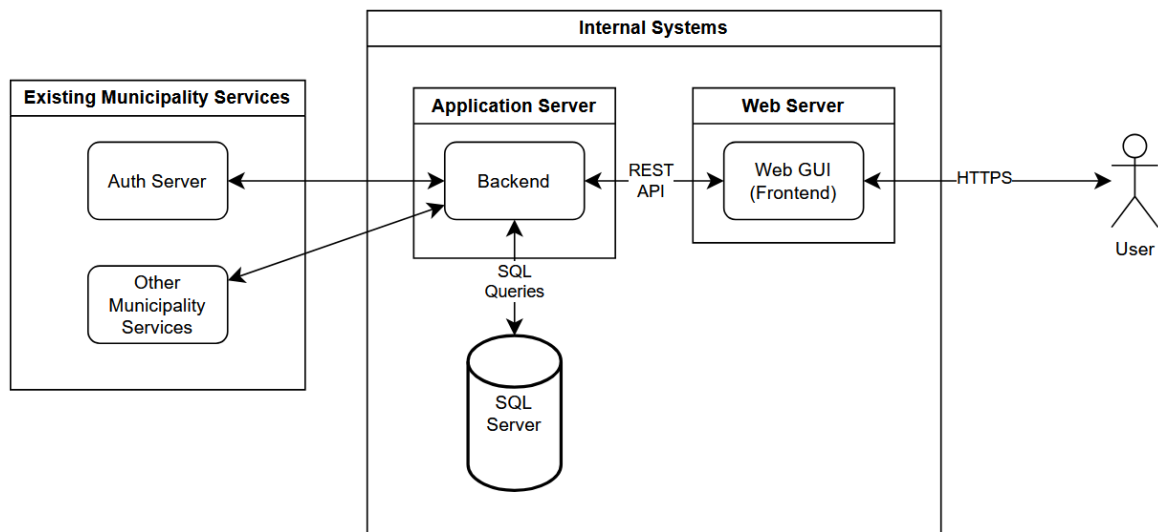
The recommended technology stack for the abandoned building management web application is:

- **Backend:** C# (.NET)
- **Frontend:** React (Mantis Dashboard, RTL-capable)

This stack ensures:

- Fast and maintainable backend development with strong integration to municipal IT infrastructure.
- Rapid frontend development using a professional, RTL-ready dashboard template.
- Consistent, user-friendly Hebrew interface tailored to municipal workers' needs.

2.2 Deployment Diagram



The system is built around a central Backend that connects all components. The User interacts with the system through the Web GUI (Frontend), which sends all requests to the Backend. The Backend performs business logic and communicates with:

- SQL Server for all data storage and retrieval.
- Auth Server (Provided by the client) to authenticate existing municipality users.
- Other Municipality Services (Provided by the client) for data integration with existing municipal systems.

The diagram shows a simple, service-oriented flow: the user works through the GUI, the GUI talks to the Backend, and the Backend handles all external system interactions.

2.3 CI/CD plan

1. Version Control and Task Management

- **Repository:** All project code will be stored in a GitHub repository.
- **Branching Strategy:**
 - main branch: stable, deployable code.
 - dev branch: integration of completed features.
 - Feature branches: created per task/issue (feature/<issue>).
- **Task Assignment:**
 - The team lead manages the GitHub repository and assigns tasks via GitHub Issues.
 - Each issue includes task description, acceptance criteria, and relevant labels.
 - Developers create feature branches linked to their assigned issues.

2. Test-Driven Development (TDD)

- **Unit Tests:** All new backend and frontend code must have unit tests written before implementation.
- **Test Frameworks:**
 1. Backend (C#): xUnit or NUnit.
 2. Frontend (Mantis/React): Jest + React Testing Library.
- **Workflow:**
 1. Write tests specifying the desired functionality.
 2. Run tests to confirm they fail initially.
 3. Implement the feature/code to make tests pass.
 4. Refactor code while ensuring all tests pass.

3. Continuous Integration (CI)

- **Trigger:** CI pipeline runs on every pull request (PR) or merge into the dev branch.
- **Steps:**
 1. Code checkout from GitHub.
 2. Dependency installation (NuGet for C#, npm/yarn for React frontend).
 3. Static code analysis for style, security, and linting.
 4. Run automated tests (unit tests and integration tests).
 5. Report results in GitHub (PR status: pass/fail).
- **Fail Policy:** Pull requests cannot be merged into the dev branch if any tests fail.

4. Continuous Deployment (CD)

- **Staging Environment:**
 - After merging into the dev branch, the CI/CD pipeline deploys the app to a staging environment for QA and user acceptance testing.
 - Staging mimics the production environment (Windows-based server for C#, web server for Mantis frontend).
- **Production Deployment:**
 - Deployment to production occurs after QA approval and team lead verification.
 - **Automated deployment** scripts handle backend services, database migrations, and frontend updates.
- **Rollback:** Versioned releases and automated backup of production database allow rollback if issues occur.

5. Team Collaboration

- **Pull Request Reviews:**
 - All code must be reviewed by at least one other developer or the team lead before merging.
- **Issue Linking:** PRs must reference the GitHub Issue ID they address.
- **Documentation:** Each feature must include comments and update relevant design/architecture documentation.

6. Summary

This CI/CD plan ensures:

- Rapid and reliable integration and deployment.
- High code quality through TDD and automated testing.
- Transparent task management via GitHub Issues.
- Smooth collaboration under the team lead's supervision.
- Minimized risk of bugs in production, especially important for managing sensitive municipal data.

3. Risk Management

3.1 Risks and mitigation

R-1 – External Integrations

There is a risk that external municipal systems (GIS, CRM-106, Water, Property Tax) may have unstable APIs, downtime, or schema changes.

Likelihood: Medium **Impact:** High

Mitigation: Build a modular integration layer that abstracts each external system. Implement retry logic, fallback batch processing, data-staleness indicators in the UI, and monitor data feed health continuously.

R-2 – Authentication / Identity Provider Delays

Integration with the municipality's identity provider (SSO and MFA) may face delays, unexpected policy constraints, or outages.

Likelihood: Medium **Impact:** High

Mitigation: Immediately schedule a kickoff meeting with the IDP/IT team to gather requirements and obtain sandbox access. Establish scoped interface contracts and early prototype login flow. Deploy a temporary fallback authentication service for internal testing until full IDP integration is verified. Set up monitoring of IDP SLA and establish escalation paths with the provider.

R-3 – System Performance and Search Load

Heavy search/filter operations on buildings and log records may degrade system response time and affect user experience.

Likelihood: Medium **Impact:** Medium

Mitigation: Design database indexes and query plans in advance for high-filter usage. Implement pagination and default limits, caching of frequent queries, and conduct early load testing with realistic volumes. Monitor key performance metrics post-deployment and tune as needed.

R-4 – Security and Permissions Configuration

Incorrect role or permission configuration could allow unauthorized edits or data access, jeopardizing data integrity and compliance.

Likelihood: Low **Impact:** High

Mitigation: Define the role model (Viewer, Editor, Admin) clearly in the requirements. Implement Role-Based Access Control (RBAC) at API gateway/middleware. Conduct a permission review and penetration test before launch, and log all permission-change actions for audit.

R-5 – Deployment and Environment Mismatch

Differences between staging and production (container versions, database config) may lead to failures after deployment.

Likelihood: Low **Impact:** Medium

Mitigation: Standardize environments using infrastructure-as-code (e.g., Docker Compose, Helm charts). Maintain version control for infrastructure. Run automated smoke tests after deployment. Ensure roll-back capability is in place.

R-6 – Project Delays and Dependencies

External dependencies (data access approvals, municipal IT coordination) may delay critical milestones and impact delivery schedule.

Likelihood: Medium **Impact:** Medium

Mitigation: Build schedule buffers for external dependency tasks. Track dependencies in the project plan and hold weekly reviews with stakeholders. Prioritize long-lead tasks (e.g., API keys, data access contracts) early and monitor progress.

3.2 POC recognition

A Proof of Concept (POC) will be developed to validate a critical functional requirement of the system: the ability to ingest existing municipal data into the new application. The POC will focus specifically on importing data from an Excel file into the system's database.

Objective: Verify that the system can reliably read, parse, and migrate abandoned-building records from legacy Excel spreadsheets into the SQL Server database that will be used in the final application.

Scope

The POC demonstrated the following:

1. Excel File Parsing

- Correct extraction of rows, columns, and data types.
- Handling of Hebrew text and right-to-left formatting.

2. Data Validation

- Identifying missing fields, incorrect formats, and invalid values.
- Ensuring required fields (e.g., building ID, address) are consistent.

3. Database Import Workflow

- Mapping Excel fields to database schema.
- Writing validated data into SQL Server tables.
- Preventing duplicate entries through primary key or unique field checks.

4. Error Handling and Reporting

- Reporting invalid rows.

Outcome

The POC successfully confirmed that:

- Existing Excel data can be used as a reliable source for initializing the new system.
- Full-scale data migration is feasible with minimal risk.
- The planned backend and database architecture supports structured data ingestion.
- Hebrew-text records are parsed and stored correctly.

3.3 MVP (Minimal Viable Product)

The MVP will replace the municipality's existing Excel-based workflow with a functional system interface for viewing abandoned building data. The primary goal of the MVP is to display the buildings list inside the system in a clear, searchable table, providing immediate value even before advanced features are implemented.

The MVP will include: A buildings list table shown in the web system, containing the key fields currently stored in the Excel spreadsheet. Loading the data into the SQL database from an imported Excel template provided by the municipality.

MVP User story: As a Municipality staff user, I want to view a table of building records in the system so that the information is current, searchable, and replaces the manual Excel process.